

Recursividade

Funções que chamam a si mesmas e a pilha de execução

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Funções que chamam a si mesmas: caso base e passo recursivo



O que a máquina guarda a cada chamada: a pilha de execução



O que se faz antes e depois da chamada: a ida e a volta



Quando a função chama a si mesma duas vezes



Recursão ou laço: o que cada um custa

Por que recursão

Muita coisa se define assim

$n! = n \cdot (n - 1)!$, e $1! = 1$

a soma de um vetor é o último elemento
mais a soma do resto

uma pasta contém arquivos e *outras pastas*

O que vamos descobrir

Em C, uma função pode chamar a si mesma
— e o código fica com a mesma cara da
definição.

Mas cada chamada tem o seu próprio n .
**Onde ficam todos esses n ao mesmo
tempo?**

A pergunta de hoje

O que a máquina faz, passo a passo, quando uma função chama a si mesma?

```
/* pilha.c --- o que acontece a cada chamada de uma funcao recursiva */
#include <stdio.h>
/* A: fatorial, contando o que acontece na ida e na volta */
long fatorial(int n)
{
    static int nivel = 0;    /* static: um so para todas as chamadas */
    printf("%sfatorial(%d) começa    &n = %p\n", 2 * nivel, "", n,
           (void *) &n);
    nivel++;
    long resultado;
    if (n <= 1)
        resultado = 1;      /* caso base */
    else
        resultado = n * fatorial(n - 1);    /* passo recursivo */
    nivel--;
    printf("%sfatorial(%d) devolve %ld\n", 2 * nivel, "", n, resultado);
    return resultado;
}
```

```
/* B: o que se faz antes e depois da chamada */
void desce_e_sobe(int n)
{
    if (n == 0)
        return;
    printf("%d ", n);      /* antes da chamada */
    desce_e_sobe(n - 1);
    printf("%d ", n);      /* depois da chamada */
}

/* C: uma funcao que chama a si mesma duas vezes */
long fib(int n, int *chamadas)
{
    (*chamadas)++;
    if (n < 2)
        return n;
    return fib(n - 1, chamadas) + fib(n - 2, chamadas);
}
```

```
/* D: a soma de um vetor e' o ultimo mais a soma do resto */
int soma(const int *v, int n)
{
    if (n == 0)
        return 0;
    return soma(v, n - 1) + v[n - 1];
}
```

Dois detalhes de printf no trecho A

`%.2s` com imprime 2 * nível espaços: é só a indentação do rastro.

`%p` imprime um endereço; por isso o `(void *) &n`.

```
int main(void)
{
    printf("A: 4! = %ld\n", fatorial(4));
    printf("B: ");
    desce_e_sobe(3);
    printf("\n");
    int chamadas = 0;
    long f = fib(5, &chamadas);
    printf("C: fib(5) = %ld com %d chamadas\n", f, chamadas);
    chamadas = 0;
    f = fib(20, &chamadas);
    printf("C: fib(20) = %ld com %d chamadas\n", f, chamadas);
    int v[] = {3, 1, 4, 1, 5};
    printf("D: soma = %d\n", soma(v, 5));
    return 0;
}
```

Antes de compilar, escreva sua resposta

1. Quantas linhas `comeca` e quantas `devolve` aparecem — e em que ordem?
2. O endereço `&n` é o mesmo nas chamadas de `fatorial`?
3. A linha **A** sai antes ou depois do rastro de `fatorial`?
4. O que a linha **B** imprime?
5. Na linha **C**, quantas chamadas faz `fib(5)`? E `fib(20)`: umas 80, umas 400 ou mais de 10 000?
6. Quanto vale a soma da linha **D**?

```
$ gcc -Wall -Wextra pilha.c -o pilha
$ ./pilha
fatorial(4) começa   &n = 0x16fb4a6fc
  fatorial(3) começa   &n = 0x16fb4a6ac
    fatorial(2) começa   &n = 0x16fb4a65c
      fatorial(1) começa   &n = 0x16fb4a60c
        fatorial(1) devolve 1
      fatorial(2) devolve 2
    fatorial(3) devolve 6
  fatorial(4) devolve 24
A: 4! = 24
B: 3 2 1 1 2 3
C: fib(5) = 5 com 15 chamadas
C: fib(20) = 6765 com 21891 chamadas
D: soma = 14
```

Os endereços na sua máquina serão outros; o padrão entre eles, não.

Caso base e passo recursivo

Função recursiva

Uma função é **recursiva** quando chama a si mesma, direta ou indiretamente. Cada chamada resolve uma *instância menor* do mesmo problema, até chegar a uma instância tão pequena que se resolve sem chamar ninguém.

Intuição

“Se a instância é pequena, resolva diretamente. Senão, reduza-a a uma instância menor do *mesmo* problema, confie que a chamada recursiva a resolve, e use a resposta.”

Fonte: Feofiloff (2009), cap. 2 [3]; Kernighan & Ritchie (1989), §4.10 [5]

$$n! = \begin{cases} 1 & \text{se } n \leq 1 \\ n \cdot (n-1)! & \text{se } n > 1 \end{cases}$$

caso base: responde sem recursão

passo recursivo: usa a resposta
de uma instância menor

```
long fatorial(int n)
{
    if (n <= 1)
        return 1;           /* caso base */
    return n * fatorial(n - 1); /* passo      */
}
```

Cada linha da definição virou uma linha do código. É essa correspondência que torna a recursão fácil de escrever — e de conferir.

Fonte: Tenenbaum et al. (1995), cap. 3 [7]; King (2008), §9.6 [6]

Por que `fatorial(1)` não chamou `fatorial(0)`?



No rastro, a descida parou em
`fatorial(1)`:

```
fatorial(4) começa
  fatorial(3) começa
    fatorial(2) começa
      fatorial(1) começa
        fatorial(1) devolve 1
```

Pense antes de passar o slide

Qual linha do código decidiu parar?

E o que garante que, começando de qualquer $n \geq 1$, essa linha *sempre* será alcançada?

Caso base

Uma instância pequena resolvida **sem** chamar a função. Aqui: `if (n <= 1)`
`resultado = 1;`

Passo recursivo

Chama a função com uma instância **estritamente menor**, que caminha em direção ao caso base. Aqui: $n \rightarrow n - 1$.

Se faltar uma das peças

Sem caso base, ou com um passo que não se aproxima dele (por exemplo `fatorial(n + 1)`), a função nunca para de se chamar. O que acontece com a máquina nesse caso é assunto da próxima aula.

Fonte: Feofiloff (2009), cap. 2 [3]; Deitel & Deitel (2011), §5.14 [2]

Exemplo: a soma da linha D



```
int soma(const int *v, int n)
{
    if (n == 0)          /* vetor vazio */
        return 0;
    return soma(v, n - 1) + v[n - 1];
}
```

$soma(v, 5) = soma(v, 4) + 5$
 $soma(v, 4) = soma(v, 3) + 1$
 $soma(v, 3) = soma(v, 2) + 4$
 $soma(v, 2) = soma(v, 1) + 1$
 $soma(v, 1) = soma(v, 0) + 3$
 $soma(v, 0) = 0$ (caso base)

Voltando: $0 + 3 = 3$, $3 + 1 = 4$, $4 + 4 = 8$, $8 + 1 = 9$,
 $9 + 5 = 14$.

Caso base: o vetor vazio, cuja soma é 0. Passo: o vetor encolhe um elemento por chamada — e n chega a 0 em exatamente 5 passos.

Fonte: Feofiloff (2009), cap. 2 [3]

O que aprendemos

- uma função recursiva chama a si mesma com uma instância menor;
- o **caso base** responde sem recursão e é onde a descida para;
- o **passo recursivo** precisa *se aproximar* do caso base a cada chamada;
- escrever a função é traduzir a definição: um caso para cada linha.

A pilha de execução

Por que `&n` mudou a cada chamada?



```
fatorial(4)    &n = 0x16fb4a6fc
fatorial(3)    &n = 0x16fb4a6ac
fatorial(2)    &n = 0x16fb4a65c
fatorial(1)    &n = 0x16fb4a60c
```

Pense antes de passar o slide

É a mesma função, com a mesma variável `n`. Por que quatro endereços diferentes?

E por que `fatorial(4)` ainda sabe que o seu `n` vale 4 quando `fatorial(3)` devolve?

Registro de ativação (*stack frame*)

A cada chamada, a máquina reserva um bloco de memória só dela, com os **parâmetros**, as **variáveis locais** e o **endereço de retorno** (onde continuar quando ela terminar).

Pilha de execução

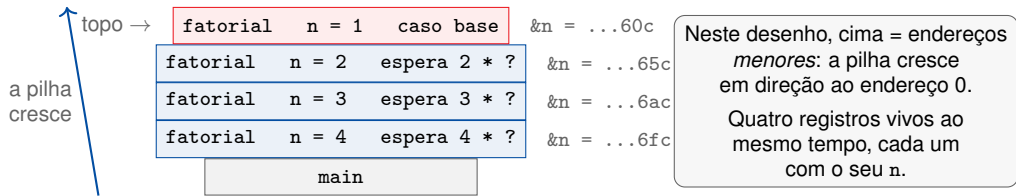
Os registros ficam empilhados: o da chamada mais recente fica no **topo** e é o primeiro a sair, quando ela retorna.

Intuição

Quatro chamadas ativas de `fatorial` são quatro registros — e, portanto, quatro variáveis `n` distintas.

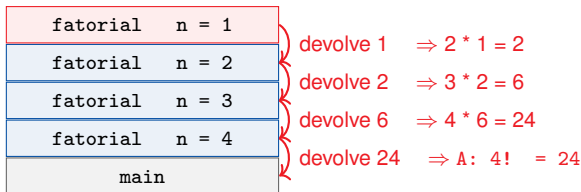
Fonte: Bryant & O'Hallaron (2015), §3.7.1 [1]; Deitel & Deitel (2011), cap. 5 [2]

A ida: cada chamada empilha um registro



Fonte: Bryant & O'Hallaron (2015), §3.7.1 e §3.7.6 [1]

A volta: cada `return` desempilha um registro



A multiplicação $n * \text{fatorial}(n - 1)$ só termina na volta: cada registro ficou esperando, com o seu n guardado, até o de cima devolver. Por isso as linhas `devolve` saem na ordem inversa das começa.

Fonte: Deitel & Deitel (2011), §5.14 [2]

Por que a distância entre os `&n` é sempre a mesma?



```
0x16fb4a6fc - 0x16fb4a6ac = 0x50
0x16fb4a6ac - 0x16fb4a65c = 0x50
0x16fb4a65c - 0x16fb4a60c = 0x50
```

`0x50` = **80 bytes** nesta máquina.

Resposta

Todas as chamadas são da mesma função, com os mesmos parâmetros e locais: o compilador reserva **o mesmo tamanho** de registro para cada uma.

Os endereços *diminuem*: na maioria das máquinas, a pilha cresce para endereços menores.

Outro compilador ou processador dá outro tamanho (48, 64, ...) — mas sempre o mesmo de uma chamada para a seguinte.

Fonte: Bryant & O'Hallaron (2015), §3.7.1 [1]

E o nível? Por que não recomeçou do zero?



O que o rastro mostra

A indentação cresceu a cada chamada e diminuiu a cada retorno. Então todas as chamadas enxergam **o mesmo** nível.

Por quê

Uma variável local `static` não mora no registro de ativação: existe uma só, durante todo o programa, e é inicializada uma única vez.

	<code>int n (parâmetro)</code>	<code>static int nivel</code>
quantos existem	um por chamada ativa	um só
onde	no registro, na pilha	fora da pilha
vida	até a chamada retornar	o programa inteiro

Fonte: Kernighan & Ritchie (1989), §4.6 [5]

Por que a linha A saiu depois de todo o rastro?

```
printf("A: 4! = %ld\n", fatorial(4));
```

Pense antes de ler a resposta

O `printf` aparece *antes* de `fatorial` nessa linha. Por que o texto A: saiu por último?

Resposta

Para chamar `printf`, primeiro é preciso o *valor* de cada argumento. `fatorial(4)` roda inteira — com todo o rastro — e só então `printf` é chamada com o 24.

Antes de uma chamada começar, todos os seus argumentos já foram calculados.

Fonte: ISO/IEC 9899:2018, §6.5.2.2 [4]

O que aprendemos

- cada chamada ganha um **registro de ativação** com parâmetros, locais e endereço de retorno;
- por isso cada chamada recursiva tem o *seu* `n`;
- o registro mais recente fica no topo e sai primeiro: a volta acontece na ordem inversa da ida;
- uma local `static` fica fora da pilha e é compartilhada por todas as chamadas;
- os argumentos são calculados antes de a chamada começar.

A ida e a volta

Por que B imprimiu 3 2 1 1 2 3?

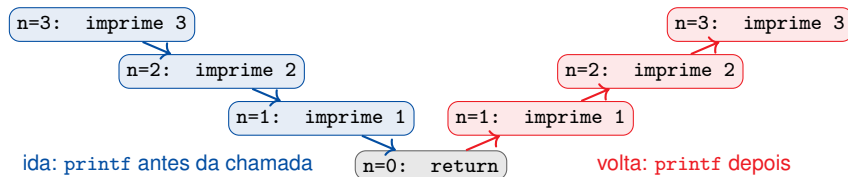
```
void desce_e_sobe(int n)
{
    if (n == 0)
        return;
    printf("%d ", n);    /* antes */
    desce_e_sobe(n - 1);
    printf("%d ", n);    /* depois */
}
```

Pense antes de passar o slide

Os dois printf são iguais. Por que o primeiro imprime em ordem decrescente e o segundo em ordem crescente?

E quantas vezes o return da linha 4 é executado?

A resposta: o que vem antes acontece na ida



O que está **antes** da chamada recursiva roda na ida, do maior para o menor. O que está **depois** só roda quando a chamada de cima retorna — na ordem inversa, como a pilha manda. Cada registro ainda guarda o seu n quando chega a vez dele.

Fonte: Tenenbaum et al. (1995), cap. 3 [7]

Exemplo: imprimir uma string de trás para frente



```
void ao_contrario(const char *s)
{
    if (*s == '\\0')           /* vazia      */
        return;
    ao_contrario(s + 1);       /* o resto... */
    putchar(*s);               /* ...e eu    */
}

int main(void)
{
    ao_contrario("pilha");     /* ahlip */
    putchar('\\n');
    return 0;
}
```

Sem índice, sem strlen, sem laço decrescente.

Na ida, cada registro guarda um ponteiro para uma letra: p, i, l, h, a.

Na volta, eles imprimem na ordem inversa: a, h, l, i, p.

Fonte: Deitel & Deitel (2011), cap. 5 [2]; Kernighan & Ritchie (1989), §4.10 [5]

O que aprendemos

- o código **antes** da chamada recursiva roda na ida, na ordem das chamadas;
- o código **depois** dela roda na volta, na ordem inversa;
- a pilha é o que torna a volta possível: cada registro ainda tem os seus valores quando chega a vez dele;
- trocar uma linha de lugar em relação à chamada muda a ordem de tudo.

Recursão múltipla

Por que `fib(5)` precisou de 15 chamadas?

$$F(n) = \begin{cases} n & \text{se } n < 2 \\ F(n-1) + F(n-2) & \text{se } n \geq 2 \end{cases}$$

C: `fib(5)` = 5 com 15 chamadas

C: `fib(20)` = 6765 com 21891 chamadas

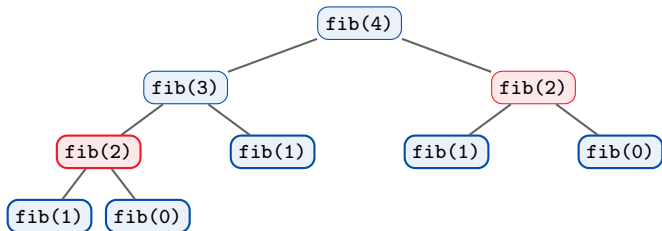
Pense antes de passar o slide

`fatorial(5)` faria 5 chamadas. Por que `fib(5)` faz 15?

E por que `fib(20)` faz mais de três vezes o próprio resultado?

Fonte: Deitel & Deitel (2011), §5.15 [2]

A árvore de chamadas de `fib(4)`



Cada chamada com $n \geq 2$ abre **duas**: 9 nós para `fib(4)`. Então $\text{fib}(5) = 1 + 9$ (de `fib(4)`) + 5 (de `fib(3)`) = **15**. E `fib(2)` é calculada duas vezes do zero.

Fonte: Deitel & Deitel (2011), §5.15 [2]; Feofiloff (2009), cap. 2 [3]

A árvore tem 9 nós, mas a pilha nunca passa de 4



pilha no momento mais fundo

fib n = 1
fib n = 2
fib n = 3
fib n = 4
main

A pilha guarda só o **caminho** da raiz até a chamada atual. Quando `fib(1)` retorna, o registro dela sai, e só então `fib(0)` entra no mesmo lugar.

Total de chamadas: cresce com a **árvore**.
Registros ao mesmo tempo: a **altura**, no máximo n .

São duas medidas diferentes: *quantas chamadas* diz quanto tempo o programa leva; *quantas ao mesmo tempo* diz quanta pilha ele usa.

Fonte: Bryant & O'Hallaron (2015), §3.7.6 [1]

n	$F(n)$	chamadas
5	5	15
10	55	177
20	6 765	21 891
30	832 040	2 692 537
40	102 334 155	331 160 281

O número de chamadas é $2F(n+1) - 1$: cresce no mesmo ritmo que a própria sequência, multiplicando-se por $\approx 1,6$ a cada n .

A pilha, enquanto isso, nunca passa de n registros. O problema aqui não é memória: é o **trabalho repetido**.

Recursão múltipla é natural quando os subproblemas são *diferentes* (as duas metades de um vetor, as subpastas de uma pasta). Quando eles se repetem, como aqui, o custo explode.

Fonte: Deitel & Deitel (2011), §5.15–5.16 [2]; Ziviani (2011), §2.2 [8]

O que aprendemos

- uma função que se chama duas vezes forma uma **árvore** de chamadas;
- o número total de chamadas pode crescer muito mais rápido que n ;
- a pilha guarda só um caminho dessa árvore: no máximo a sua altura;
- tempo (total de chamadas) e pilha (chamadas simultâneas) são custos distintos.

Recursão ou laço?

Recursivo

```
long fatorial(int n)
{
    if (n <= 1)
        return 1;
    return n * fatorial(n - 1);
}
```

n registros na pilha, n chamadas.

Iterativo

```
long fatorial(int n)
{
    long r = 1;
    for (int i = 2; i <= n; i++)
        r *= i;
    return r;
}
```

Um registro, uma chamada; o laço muda r e i no lugar.

Toda função recursiva pode ser reescrita com laço (às vezes com uma pilha explícita). A recursiva costuma ser mais próxima da definição; a iterativa, mais econômica.

Fonte: Deitel & Deitel (2011), §5.16 [2]; Tenenbaum et al. (1995), cap. 3 [7]

Prefira recursão quando

- o problema se divide em partes do mesmo tipo (pastas, árvores, metades de vetor);
- a versão com laço precisaria de uma pilha feita à mão;
- a profundidade é pequena.

Prefira o laço quando

- cada chamada faz só uma chamada, no fim (fatorial, soma);
- os subproblemas se repetem (fib ingênua);
- a profundidade pode ser enorme.

Próxima aula: e `soma(v, 10000000)`? Dez milhões de registros de 80 bytes cabem na pilha?

Fonte: Deitel & Deitel (2011), §5.16 [2]; Feofiloff (2009), cap. 2 [3]

O que aprendemos

- recursão e laço resolvem os mesmos problemas;
- a recursão paga um registro de ativação por chamada ativa;
- ela compensa quando a estrutura do problema já é recursiva;
- o laço compensa quando a recursão é só uma contagem disfarçada.

Mãos à obra

Tarefa 1

Troque o caso base de `fatorial` para `n == 0`. Quantas linhas `comeca` aparecem? E `fatorial(0)`, nas duas versões?

Tarefa 2

Em `desce_e_sobe`, apague um `printf` de cada vez e preveja a saída. Depois faça a função imprimir 3 2 1 0 1 2 3.

Tarefa 3

Faça `fib` medir também a profundidade máxima da pilha. Quantos registros `fib(20)` empilha ao mesmo tempo?

Potência em poucas chamadas

Escreva duas versões de `long potencia(long x, int n)`:

- **ingênua:** $x^n = x \cdot x^{n-1}$, com $x^0 = 1$;
- **rápida:** $x^n = (x^{n/2})^2$ se n é par, e $x \cdot x^{n-1}$ se é ímpar — calculando $x^{n/2}$ *uma vez só*.

Requisitos:

- as duas devolvem o mesmo valor para $2^0, \dots, 2^{62}$;
- conte as chamadas de cada uma para `potencia(1, 1000)` e explique a diferença;
- desenhe a pilha da versão rápida no momento mais fundo de `potencia(2, 10)`.

Fonte: Ziviani (2011), §2.2 [8]

O que mudou hoje

Uma função que chama a si mesma deixou de ser mágica: é uma chamada como outra qualquer, com o seu próprio registro na pilha.

O preço

Cada chamada ativa ocupa um registro. Enquanto a descida não chega ao caso base, nenhum deles é devolvido.

Para levar

- caso base para parar, passo recursivo que se aproxima dele;
- um registro por chamada: parâmetros, locais, endereço de retorno;
- antes da chamada roda na ida; depois dela, na volta;
- chamadas no total $\neq$ chamadas ao mesmo tempo.

-  Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seção 3.7: procedimentos, a pilha de execução e procedimentos recursivos.
-  Paul Deitel and Harvey Deitel.
C: como programar.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.
Capítulo 5: pilha de chamadas, recursão, Fibonacci e recursão versus iteração.

-  Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 2: algoritmos recursivos.
-  ISO/IEC.
ISO/IEC 9899:2018 — Information technology — Programming languages —
C.
Technical report, International Organization for Standardization, 2018.
Seção 6.5.2.2: chamadas de função.



Brian W. Kernighan and Dennis M. Ritchie.

C: a linguagem de programação — padrão ANSI.

Campus, Rio de Janeiro, 1989.

Seções 4.6 (variáveis static) e 4.10 (recursividade).



K. N. King.

C Programming: A Modern Approach.

W. W. Norton, New York, 2 edition, 2008.

Seção 9.6: recursion.

-  Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
Capítulo 3: recursividade.
-  Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.
Seção 2.2: recursividade.